

langchain-ai/langgraph

LangGraph : socle modulaire, MIT, tenu par une entreprise et peu de mains — adopter oui, dépendre aveuglément non

Build resilient agents.

4

BUS FACTOR
sur 12 semaines

112

COMMITTS
sur 12 semaines

33

DÉPENDANCES
6 en production

MIT

LICENCE

Que fait ce projet et quelle est son architecture ?

Neuf paquets séparés, un cœur MIT de 6 dépendances : on adopte LangGraph par morceaux, pas en bloc.

<code>.github/</code>	17 workflows GitHub Actions, scripts CI
<code>docs/</code>	documentation
<code>examples/</code>	exemples d'usage, notebooks
<code>libs/</code>	code principal : 9 paquets, 8 en Python, sdk-js réduit à un README
<code>.gitignore</code>	exclusions git
<code>.markdownlint.json</code>	config lint Markdown
<code>AGENTS.md</code>	consignes pour les agents IA
<code>CLAUDE.md</code>	consignes pour Claude Code
<code>LICENSE</code>	licence MIT
<code>Makefile</code>	pilote du monorepo : install, lint, test via uv
<code>README.md</code>	présentation du projet

LangGraph est un framework d'orchestration bas niveau pour agents à état : exécution durable, reprise après panne exactement au point d'arrêt, interruption pour validation humaine, mémoire courte et longue.

Ce n'est pas un produit clé en main mais une couche d'infrastructure : le modèle raisonne, LangGraph tient l'état, la reprise et le contrôle.

L'architecture est un monorepo de neuf paquets sous `libs/`, publiés séparément : le cœur, quatre paquets de persistance, les briques préconstruites, la CLI et deux SDK.

Le cœur tient dans `libs/langgraph/langgraph` — `graph/`, `pregel/`, `runtime.py` — et se distribue en wheel MIT versionnée 1.2.11.

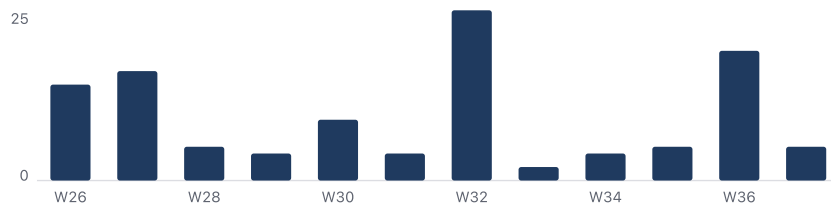
Conséquence directe pour une adoption : la persistance est un point d'extension, pas un choix imposé. Postgres, SQLite et une interface de conformité sont des paquets distincts, remplaçables par le vôtre.

Le monorepo est piloté par un `Makefile` racine et `uv` : `make install` crée l'environnement et installe chaque lib en editable, `make lint`, `make lock` et `make test` délèguent à chaque paquet.

Le modèle d'exécution est emprunté à Pregel et Apache Beam, l'interface publique à NetworkX : rien d'exotique, des fondations documentées depuis quinze ans.

Quelle est sa santé technique ?

112 modifications en 12 semaines mais 23 des 33 intégrations du mois sont des bumps : le projet est entretenu plus qu'il n'accélère.



112 modifications sur les douze dernières semaines, la dernière la veille du relevé : le projet est actif, sans ambiguïté.

Le rythme est irrégulier — de 2 à 25 modifications par semaine — signature d'un développement par vagues autour des livraisons, pas d'un essoufflement.

33 propositions de changement ont été intégrées en trente jours, mais 23 d'entre elles sont des montées de version de dépendances : le débit réel de fonctionnalités est de l'ordre de dix changements par mois.

Dix livraisons en deux mois, réparties sur les paquets : le cœur est passé de 1.2.9 (10 juillet 2026) à 1.2.11 (11 août 2026), la plus récente relevée est le SDK Python 0.4.4 du 27 août 2026.

À noter pour un plan de montée de version : le cœur n'a pas été republié depuis un mois au moment du relevé, alors que les SDK et checkpointers bougent toutes les deux semaines.

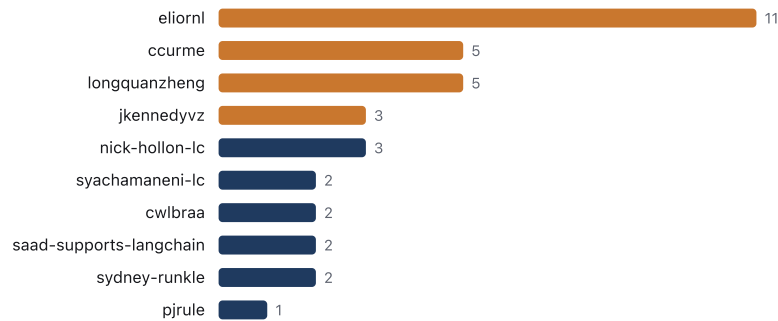
L'intégration continue est sérieuse : `ci.yml` filtre par chemins modifiés, puis lint et tests sur huit paquets Python, vérification du schéma de la CLI, tests d'intégration CLI et SDK, job agrégateur `ci_success` ; les actions sont épinglées par SHA et les exécutions concurrentes annulées.

Les tests se lancent par `make test`, en `pytest`, avec `--strict-markers --strict-config` et des snapshots `syrupy`.

Donnée non relevée : le taux de couverture et le nombre de fichiers de test, aucun outil ne les expose ici ; à vérifier dans <https://github.com/langchain-ai/langgraph/tree/main/libs/langgraph/tests>.

Qui porte le projet et quel est le bus factor ?

Un éditeur solide derrière, mais quatre personnes couvrent la moitié du travail et aucune zone n'a de responsable désigné.



Bus factor relevé : 4. Quatre départs simultanés suffisent à faire décrocher le rythme.

La distribution est plate et mince : la personne la plus active signe 11 modifications sur douze semaines, la deuxième et la troisième 5 chacune.

Personne n'est à plein temps sur le cœur visible : les dix noms relevés totalisent 36 des 112 modifications de la période, le reste vient d'une traîne non relevée nominativement.

L'éditeur est une organisation, LangChain Inc, pas un individu ; plusieurs comptes actifs portent d'ailleurs un suffixe maison (`nick-hollon-lc` , `saad-supports-langchain`).

Aucun fichier CODEOWNERS : personne n'est désigné responsable d'une zone du code, ni pour la relecture, ni pour l'astreinte.

Parade : ne jamais dépendre d'une correction amont. Prévoyez une personne capable de lire `pregel/` et de tenir un fork temporaire pendant six semaines.

Quelles dépendances et quelle dette ?

Six dépendances à l'exécution dont trois internes : la dette n'est pas dans les paquets, elle est dans le couplage à langchain-core .

Le manifeste du cœur déclare 33 dépendances, dont 27 servent au développement : pytest et ses plugins, ruff, ty, jupyter.

À l'exécution il n'en reste que six, et trois sont des paquets internes du monorepo :

DÉPENDANCE	VERSION	NATURE
langchain-core	>=1.4.7,<2	externe, couplage structurant
pydantic	>=2.7.4	externe, standard
xxhash	>=3.5.0	externe, hachage
langgraph-checkpoint	>=4.1.0,<5.0.0	interne au dépôt
langgraph-sdk	>=0.4.2,<0.5.0	interne au dépôt
langgraph-prebuilt	>=1.1.0,<1.2.0	interne au dépôt

Trois dépendances externes seulement : la surface d'attaque et le risque de conflit de versions sont faibles pour un projet de cette taille.

La vraie dépendance est langchain-core , borné à la majeure 1 : votre montée vers langchain-core 2 sera conditionnée par le calendrier de LangChain, pas par le vôtre.

Pas de manifeste racine, chaque paquet a son pyproject.toml et son uv.lock versionné (712 Ko pour le cœur) : les versions sont verrouillées, reproductibles, et montées par lots.

Dette relevée dans le code : quatre TODO structurels sur le cœur — signatures config / store à remplacer par un argument de contexte dans graph/_node.py , gestionnaire de contexte du runtime.py , paramètre exiting explicite dans pregel/_loop.py — plus un # TODO: add more tests dans tests/test_algo.py .

Donnée non relevée : les dépendances transitives et les alertes de vulnérabilité ; à vérifier dans <https://github.com/langchain-ai/langgraph/security>.

Quels risques ?

Le code et la licence ne posent pas de problème ; ce sont l'absence de canal de sécurité, quatre porteurs et 528 tickets ouverts qui doivent entrer dans votre plan.

- license** MIT : licence permissive.
- security** Pas de SECURITY.md : aucun canal déclaré pour signaler une faille.
- deps** Pas de manifeste racine : chaque paquet de libs/ a son pyproject.toml et son uv.lock. libs/langgraph/pyproject.toml déclare 33 dépendances : 6 à l'exécution (langchain-core, pydantic, xxhas...
- ci** 17 workflows dans .github/workflows. ci.yml (workflow_dispatch, push sur main, pull_request) filtre par chemins modifiés puis enchaîne lint et tests sur 8 libs Python via _lint.yml, _test.y...
- security** Aucune mesure de couverture : pytest-cov est bien dans le groupe test de libs/langgraph/pyproject.toml, mais ni ci.yml ni _test_langgraph.yml ne produisent de rapport ni n'imposent de seuil...

Quatre risques ont été évalués, un seul sort du vert.

RISQUE	NIVEAU	CONSTAT	PARADE
Licence	FAIBLE	MIT, produit fermé autorisé	garder la mention de copyright
Sécurité	MOYEN	pas de SECURITY.md	passer par le support de l'éditeur
Dépendances	FAIBLE	6 à l'exécution, uv.lock versionné	épingler le cœur, rejouer vos tests
Intégration continue	FAIBLE	17 workflows, actions épinglées par SHA	aucune, c'est un point fort

Deux risques absents de cette grille pèsent plus lourd pour une mise en production.

Concentration humaine, niveau élevé. Bus factor de 4, aucun CODEOWNERS : la parade est interne — une compétence maison sur le cœur et la capacité à tenir un fork temporaire.

File d'attente du support, niveau élevé. 528 tickets ouverts contre 7 refermés en trente jours, 245 propositions en attente : budgétez votre propre support plutôt que d'attendre l'amont.

Les tickets les plus discutés sont des sujets de production : réexécution silencieuse d'appels longs depuis un checkpoint, ordre de persistance non garanti en durability="sync" ^{1 2}.

Adopter, contribuer ou forker ?

Adopter en épinglant, contribuer en ciblé, ne pas forker : le dépôt absorbe mal les contributions mais l'amont va dans votre sens.

Adopter est le choix par défaut, et il est peu risqué juridiquement : MIT, `pip install -U langgraph`, cœur en 1.x donc engagement de stabilité de l'interface.

Le coût d'adoption réel n'est pas l'installation mais l'exploitation : les trois tickets les plus lourds portent sur la sémantique de reprise après panne, exactement ce que vous achetez en choisissant ce framework².

Contribuer est possible mais lent : 245 propositions de changement sont ouvertes, dont des correctifs sur le cœur toujours en attente, et aucun ticket n'est étiqueté « bonne première contribution ».

Le dépôt est en revanche bien outillé pour contribuer vite, y compris avec des agents : `AGENTS.md`, `CLAUDE.md` et `docs/llms.txt` sont présents, et deux workflows imposent le format des propositions et le lien vers un ticket.

Forker ne se justifie pas : rattraper 112 modifications par trimestre sur neuf paquets coûte plus cher qu'un correctif ciblé remonté en amont.

La demande de la communauté dessine la trajectoire, et elle est alignée avec des besoins d'entreprise : preuves auditable de fin d'exécution (64 échanges), exemple de validation humaine en environnement réglementé, interception des appels d'outil avant exécution pour appliquer une politique^{3 4 5}.

Donnée non relevée : aucune feuille de route publiée ni jalon daté ; à suivre dans <https://github.com/langchain-ai/langgraph/milestones>.

Décision

1. **Adopter, en épinglant le cœur.** Figez `langgraph` 1.2.11 et le checkpointer, rejouez vos tests à chaque montée : 23 des 33 intégrations du mois sont des bumps, le flux est constant.
2. **Budgéter le support en interne.** 528 tickets ouverts pour 7 refermés en trente jours : personne ne corrigera votre incident à votre place.
3. **Une compétence maison sur `pregel`.** Bus factor de 4, aucun CODEOWNERS : il faut une personne capable de lire le cœur et de tenir un fork temporaire six semaines.
4. **Deux points à instruire avant la production.** L'ordre de persistance en `durability="sync"` et la perte possible d'une exécution avant le premier checkpoint durable conditionnent votre promesse de reprise^{2 6}.
5. **Un point de vigilance non technique.** Aucun canal de divulgation de faille n'est publié : décidez dès maintenant par où vous signalerez une vulnérabilité.

RÉFÉRENCES PUBLIQUES

Sources

1. <https://github.com/langchain-ai/langgraph/issues/7417>
2. <https://github.com/langchain-ai/langgraph/issues/8039>
3. <https://github.com/langchain-ai/langgraph/issues/7844>
4. <https://github.com/langchain-ai/langgraph/issues/7687>
5. <https://github.com/langchain-ai/langgraph/issues/8102>
6. <https://github.com/langchain-ai/langgraph/issues/8764>